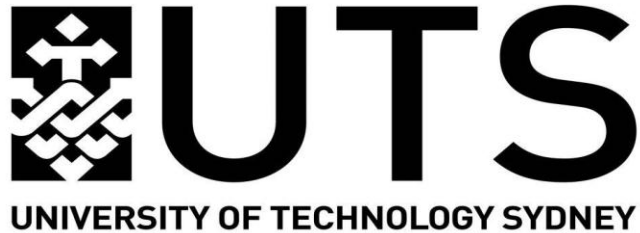


UNIVERSITY OF TECHNOLOGY SYDNEY



32513 31005 Advanced Data Analytics Algorithms, Machine Learning - Spring 2025
Assessment 2: Machine Learning Model Study / Project Implementation

Student name: Thai Minh Huong Dinh

Student ID: 25504554

Link colab: <https://colab.research.google.com/drive/1eXpA3m1GxtEEytXJ9ve4wnLXNFUDdOF?usp=sharing>

Table of Contents

Abstract	3
Motivation for the Study	3
1. Dataset Description	4
1.1 Data Overview	4
1.2 Data Type Standardization	4
1.3 Missing values	5
1.4 Duplicates & class balance	6
1.5 Outlier Handling	7
2. Problem Definition (Task + I/O Contract)	9
2.1 Definition	9
2.2 I/O Contract	9
2.3. Training/evaluation contract (Train/Val/Test)	11
2.4 Overall Architecture	12
2.5 Data Preprocessing	13
3. Model and algorithm	16
3.1 Baseline Model — Logistic Regression (Theory → Implementation)	16
3.1.1 Algorithm	16
3.1.2 Theory and code (scikit-learn)	17
3.1.3 Analysis of Model Tuning Results (Hyperparameter C)	17
3.1.4 Results	18
3.2 Advanced Model — XGBoost (Theory → Implementation)	21
3.2.1 Theory summary	21
3.2.2 Theory ↔ Code mapping	22
3.2.3 Results	23
4. Results and Evaluation	25
5. Model Explainability	26
Reference	31
Appendix	32
Appendix — Cost-based threshold analysis (Validation → Test)	32
Appendix – Ablation Logistic Regression with L2 vs L1 penalty	33
Appendix – Correlation Matrix	33
Appendix: XGBoost Model Training with Early Stopping	35
Appendix – Artifact (included with the submission)	35

Abstract

This study aims to predict credit risk using tabular borrower and loan attributes by developing an end-to-end machine learning pipeline. The pipeline involves preprocessing steps including label encoding for categorical variables, data splitting into train/validation/test sets, and model training using XGBoost (XGB) with early stopping and hyperparameter tuning. The model was optimized using the AUC-PR metric on the validation set, achieving a best validation AUC-PR score of 0.9055 at boosting round 473. For model simplicity and interpretability, an artifact version of XGB was also implemented using default parameters and label encoding, achieving an F1 score of 0.8378 on the test set. The slight variation between the tuned and simple XGB versions demonstrates the trade-off between model optimization and implementation simplicity. Overall, the XGB model shows strong potential for real-world credit risk prediction tasks.

Motivation for the Study

I select the credit-risk default dataset to build a practical, end-to-end system precisely because it mirrors real problems I will face in industry: making high-stakes, cost-sensitive decisions under uncertainty. Credit screening is a core analytics use case across banking, fintech, BNPL, and lending operations, where models must handle class imbalance, mixed tabular features (categorical + numeric), and strict requirements for fairness, interpretability, and governance. Working with this dataset develops job-ready skills along the full pipeline, data integrity checks (missing values, outliers, duplicates), leakage-safe preprocessing with pipelines, stratified splitting, threshold tuning to business costs (FN vs FP), and comparative model evaluation (baseline LR/advanced XGB with calibrated probabilities). It also supports explainability (feature importance/SHAP) for stakeholders and compliance, and it is straightforward to deploy as batch scoring or a simple scoring function for real applications. In short, this study is intentionally scoped to produce a portfolio, ready artifact that is both technically sound and directly transferrable to credit underwriting, fraud triage, collections prioritisation, and broader risk analytics roles, making it an excellent bridge between coursework and professional practice.

1. Dataset Description

1.1 Data Overview

The dataset contains 32,581 loan records, each representing an individual customer applying for credit. It includes 12 features covering personal information (e.g., age, income, employment length, home ownership, credit history), loan-related attributes (e.g., loan amount, interest rate, loan grade, loan purpose), and a derived feature (loan_percent_income) indicating the share of income allocated to loan repayment. The target variable is loan_status (0 = non-default, 1 = default), making this a binary classification problem where the goal is to predict whether a customer will default on their loan.

Mounted at /content/drive
(32581, 12)

	person_age	person_income	person_home_ownership	person_emp_length	loan_intent	loan_grade	loan_amnt	loan_int_rate	loan_status	loan_percent_income	cb_person_default_on_file	cb_person_cred_hist_length
0	22	59000	RENT	123.0	PERSONAL	D	35000	16.02	1	0.59	Y	3
1	21	9600	OWN	5.0	EDUCATION	B	1000	11.14	0	0.10	N	2
2	25	9600	MORTGAGE	1.0	MEDICAL	C	5500	12.87	1	0.57	N	3
3	23	65500	RENT	4.0	MEDICAL	C	35000	15.23	1	0.53	N	2
4	24	54400	RENT	8.0	MEDICAL	C	35000	14.27	1	0.55	Y	4

1.2 Data Type Standardization

Several features were initially stored as object in the raw dataset, although they are categorical by nature. To ensure correct handling in both exploratory analysis and modeling, these columns were standardized:

- person_home_ownership, loan_intent, loan_grade, cb_person_default_on_file were converted to category.
- For cb_person_default_on_file, values "Y" and "N" were mapped to binary (1/0) for easier use in machine learning models.
- loan_grade was treated as an *ordinal categorical variable* ($A < B < C < \dots$), preserving its natural order.

This standardization step was performed before train/test splitting as it does not involve learning from the data and helps ensure consistent downstream preprocessing.

```
person_age          int64
person_income       int64
person_home_ownership category
person_emp_length   float64
loan_intent          category
loan_grade          category
loan_amnt           int64
loan_int_rate       float64
loan_status         int64
loan_percent_income float64
cb_person_default_on_file int8
cb_person_cred_hist_length int64
dtype: object
```

Check missing values

```
def missing_report(df: pd.DataFrame):
    cnt = df.isna().sum()
    pct = (cnt/len(df)*100).round(2)
    rep = pd.DataFrame({'missing_count': cnt, 'missing_pct': pct}).sort_values('missing_count', ascending=False)
    return rep[rep.missing_count > 0]

missing_report(df)
```

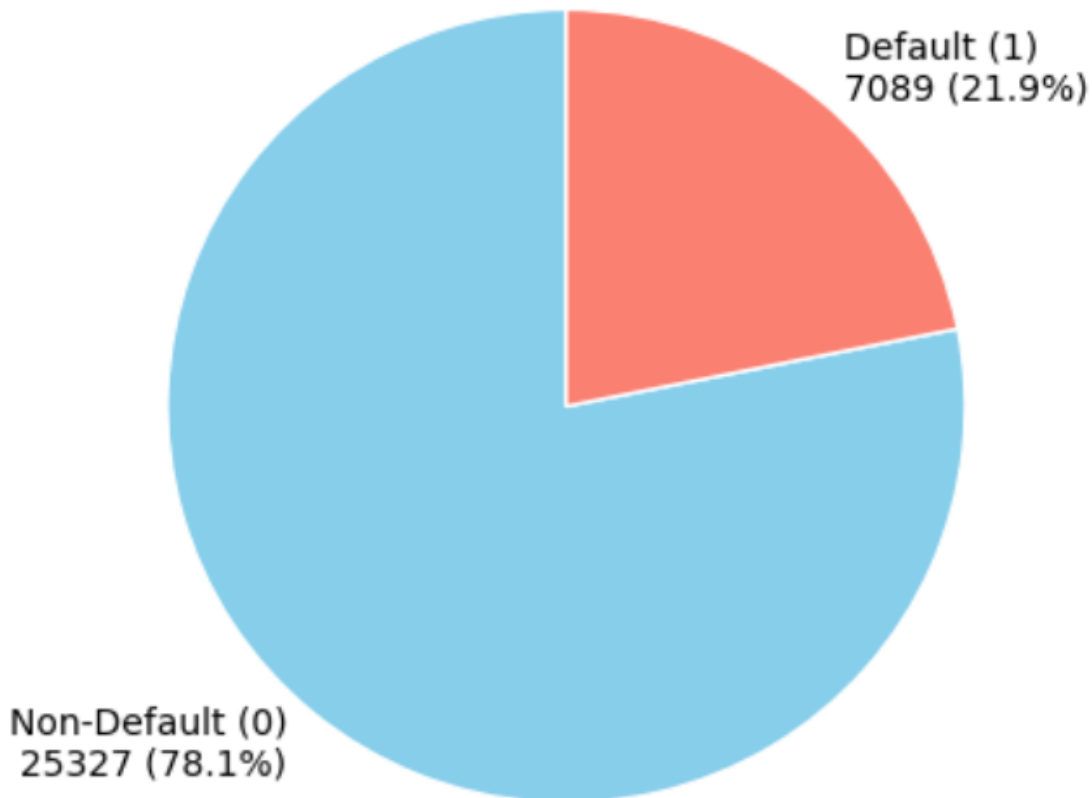
	missing_count	missing_pct
loan_int_rate	3116	9.56
person_emp_length	895	2.75

1.3 Missing values

Two fields contain missing data: loan_int_rate = 3,116 rows (9.56%) and person_emp_length = 895 rows (2.75%); all other columns are complete. To avoid information leakage, imputation will be performed after a stratified train/test split, fitting on the training set and applying the same transform to the test set. Specifically, I plan to impute with the median for loan_int_rate (robust to skewed distributions) and median (or KNN/iterative if needed) for person_emp_length (then optionally round to whole years).

```
Total duplicated rows (exact duplicates): 330
Dropped 165 rows. New shape: (32416, 12)
(32416, 12)
```

Class Balance after de-dup

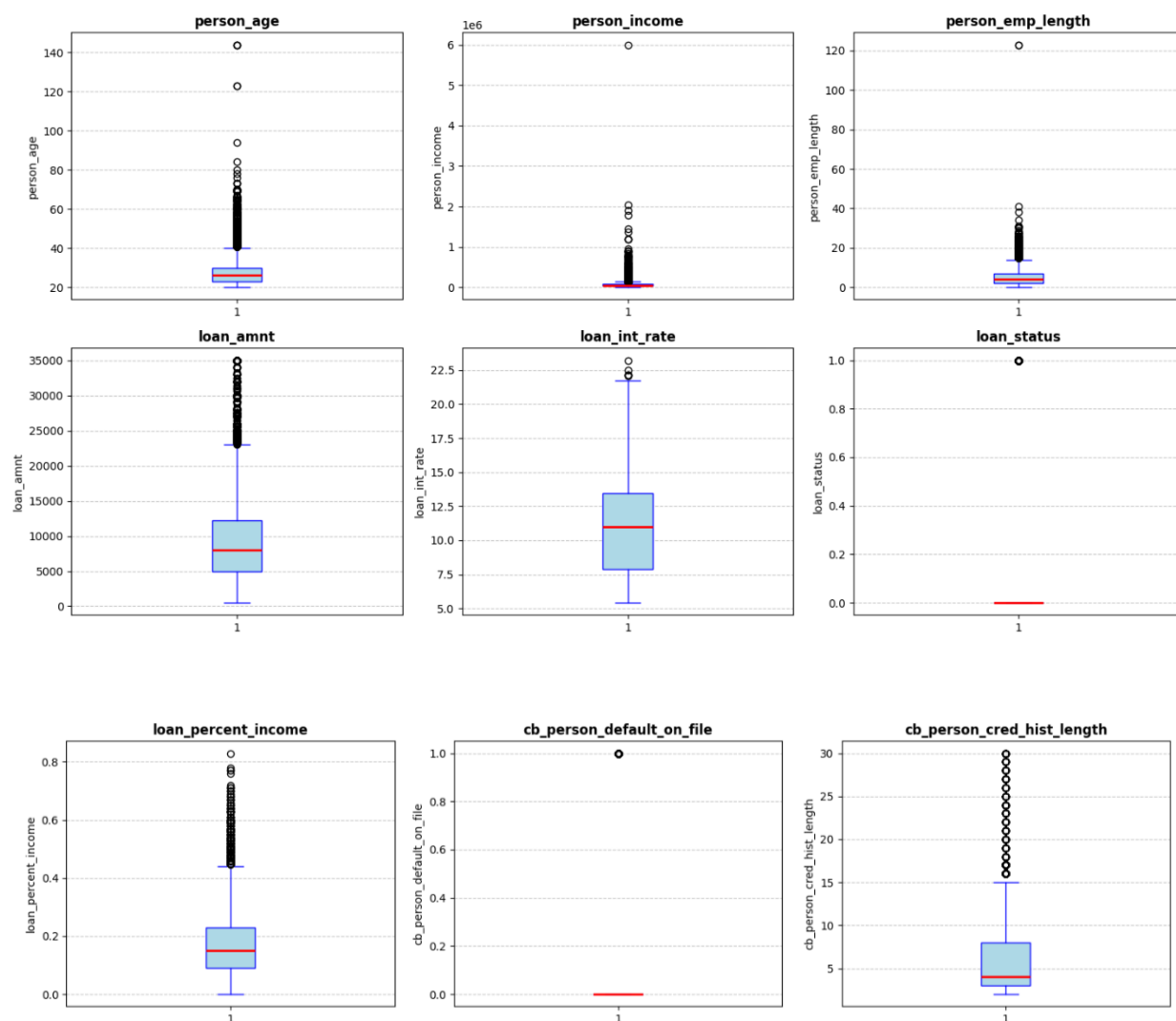


1.4 Duplicates & class balance

The duplicate check identified 330 rows belonging to duplicate groups (counting both originals and copies). After removing duplicates with `drop_duplicates(keep='first')`, the dataset decreased by 165 rows, leaving 32,416 rows $\times$ 12 columns. De-duplication was performed before the train/test split to prevent information leakage and biased evaluation. The target distribution after de-duplication is essentially unchanged: Non-Default = 25,327 (78.1%), Default = 7,089 (21.9%). This confirms the dataset is class-imbalanced, so the evaluation will prioritize imbalance-aware metrics (ROC-AUC, PR-

AUC, Recall/F1) and consider techniques such as **class weights**, **threshold tuning**, or **SMOTE**.

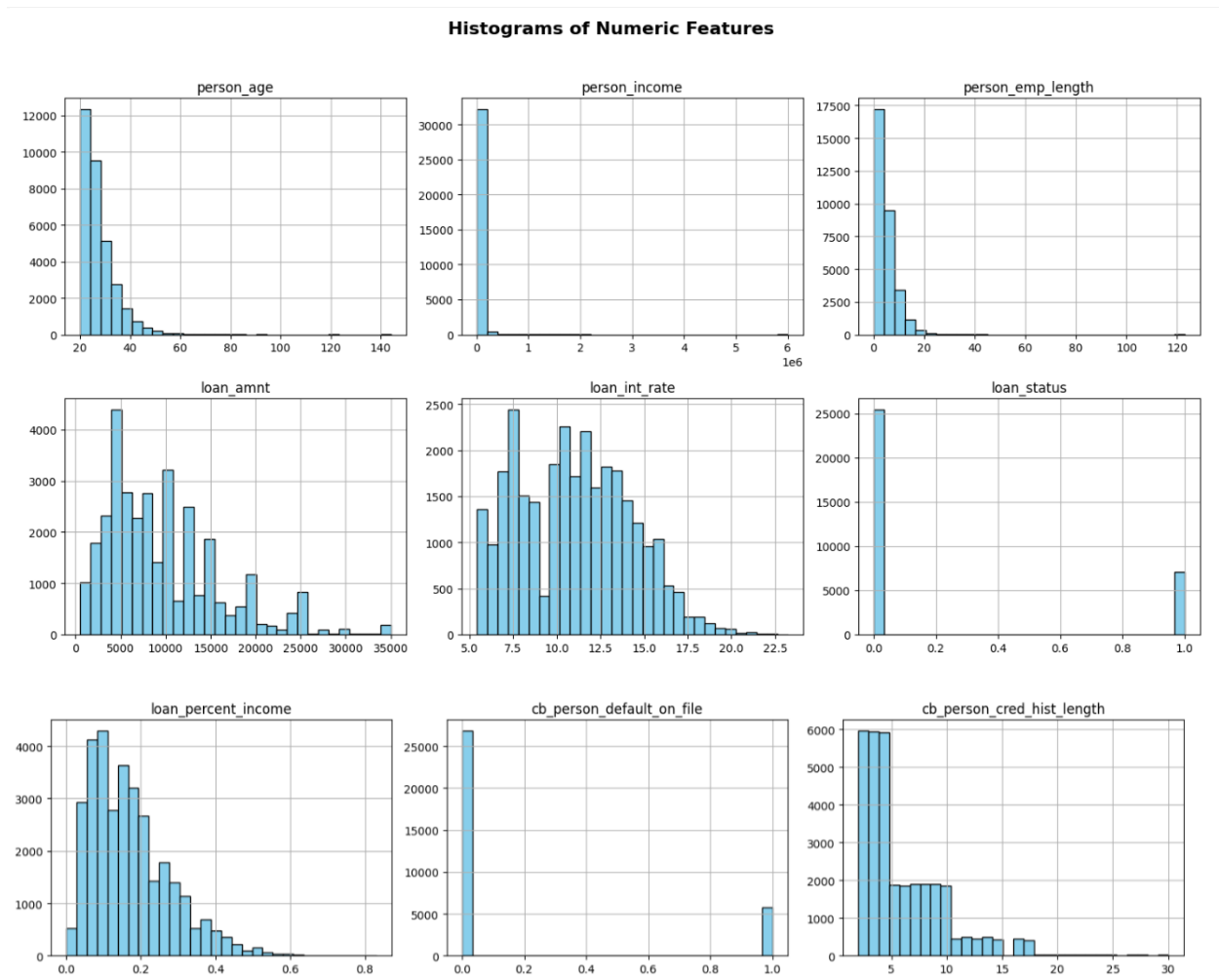
1.5 Outlier Handling



The dataset contains several numeric features (`person_age`, `person_income`, `loan_amnt`, `loan_int_rate`, `loan_percent_income`) that show a long-tailed distribution and a few extreme values far from the main bulk of data. Such outliers may arise either from genuine but rare borrower profiles (such as very high income) or from data entry errors or unusual measurement artifacts.

Because our project focuses on building a practical baseline credit-risk classifier, we chose an application-oriented approach rather than a complex outlier-specific study:

- Logistic Regression (LR) is sensitive to extreme numeric values because it assumes a roughly linear log-odds relationship; a few extreme points can distort the fitted coefficients.
- Tree-based models such as XGBoost are much less sensitive to outliers because they split on thresholds and do not rely on a global linear fit.
- To keep the comparison between models fair, we adopted a uniform data preprocessing policy.



Distribution of numeric features. Most features are **right-skewed**. `person_age` concentrates in **20–35** with a few unusually high values (>60 , even >100). `person_income` is **heavy-tailed**: the majority are $<200k$, with a few very large outliers ($\approx$ millions). `person_emp_length` is mainly **0–10 years**, with scattered values >40 .

loan_amnt shows multiple modes around common product thresholds (~5k, 10k, 15k, 20k+). loan_int_rate is fairly spread over 6–18%, peaking around 10–13%. loan_percent_income clusters at 10–25% with a long right tail >40% (higher risk). cb_person_cred_hist_length is mostly 2–8 years, with some >20. The two binary variables, loan_status and cb_person_default_on_file, confirm class imbalance (majority 0), consistent with the class balance (~78% non-default, ~22% default).

Preprocessing implications. We will impute loan_int_rate (~9.6% missing) with the median after the split. For skewed/heavy-tailed features (person_income, loan_amnt, loan_percent_income), consider log transforms or winsorizing/capping. For variables with extreme values (person_age, person_emp_length, cb_person_cred_hist_length), apply business caps (e.g., age 18–75) or IQR-based capping. Do not treat outliers for the two binary variables; instead, handle imbalance during modeling (class weights, threshold tuning, or SMOTE). All preprocessing steps will be fit on the training set and applied consistently to the test set.

2. Problem Definition (Task + I/O Contract)

2.1 Definition

The problem is a binary classification task: to predict the probability that a customer will default on a loan based on tabular demographic and loan-related features. In deployment, the system takes a new applicant's record, processes it through the standard pipeline, estimates the probability $\hat{P} = P(y = 1|x)$, and compares it to a decision threshold t^* chosen on the Validation set to assign the final label Default / Not-Default.

2.2 I/O Contract

Component (Train / Deploy)	Type	Valid range & Notes	Pre-processing
person_age	numeric (int)	clip to [18, 95]	impute (<i>median</i>) → RobustScaler
person_income	numeric (int)	as is (no clipping)	impute (<i>median</i>) → RobustScaler
person_emp_length	numeric	clip to [0, 50]	impute (<i>median</i>) →

Component (Train / Deploy)	Type	Valid range & Notes	Pre-processing
	(int)		RobustScaler
person_home_ownership	categorical	{RENT, MORTGAGE, OWN, OTHER}	impute (<i>most_frequent</i>) → One-Hot (<i>handle_unknown="ignore"</i>)
person_education	categorical	{HS, College, Bachelor, Master, Doctor, Other}	impute (<i>most_frequent</i>) → One-Hot
loan_intent	categorical	{DEBTCONSOLIDATION, EDUCATION, MEDICAL, PERSONAL, VENTURE, HOMEIMPROVEMENT}	impute (<i>most_frequent</i>) → One-Hot
loan_grade	categorical	{A, B, C, D, E, F, G} (<i>treated as nominal</i>)	impute (<i>most_frequent</i>) → One-Hot
loan_amnt	numeric (int)	as is (range from data; no clipping)	impute (<i>median</i>) → RobustScaler
loan_int_rate	numeric (float)	clip [0, 45] ; may contain missing values	impute (<i>median</i>) → RobustScaler
loan_percent_income	numeric (float)	clip [0, 1.5] (loan-to-income ratio)	impute (<i>median</i>) → RobustScaler
cb_person_default_on_file	binary	{Y, N} → {1, 0}	passthrough (<i>no scaling</i>)
cb_person_cred_hist_length	numeric (int)	clip [0, 50]	impute (<i>median</i>) → RobustScaler
Output (Model) $\hat{p}$	float	probability of default in $([0,1])$	used for PR/ROC & thresholding
Output (Decision) ($\hat{y}$)	label {0,1}	$\hat{y} = 1 \text{ if } \hat{p} \geq t^*, \text{ else } 0$	(t^*) chosen on Validation , locked for Test/Deploy

Anti-leakage principle: All fit steps (imputer, scaler, encoder, winsorization, log-transform) are learned only on the TRAIN split. These fitted transformers are then applied to the Validation, Test, and any production data. No Validation or Test data are used for fitting preprocessing, hyper-parameters, or thresholds before final model locking.

2.3. Training/evaluation contract (Train/Val/Test)

- **Split:** Stratified Train/Validation/Test ($\approx 64/16/20$).

Split train/test

```
[16]
✓ Os
from sklearn.model_selection import train_test_split

TARGET = "loan_status"
X = df.drop(columns=TARGET)
y = df[TARGET]

# 1. Split 20% test
X_rest, X_test, y_rest, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)

# 2. Split 20% of remaining 80% = 16% for validation
X_train, X_val, y_train, y_val = train_test_split(X_rest, y_rest, test_size=0.2, stratify=y_rest, random_state=42)

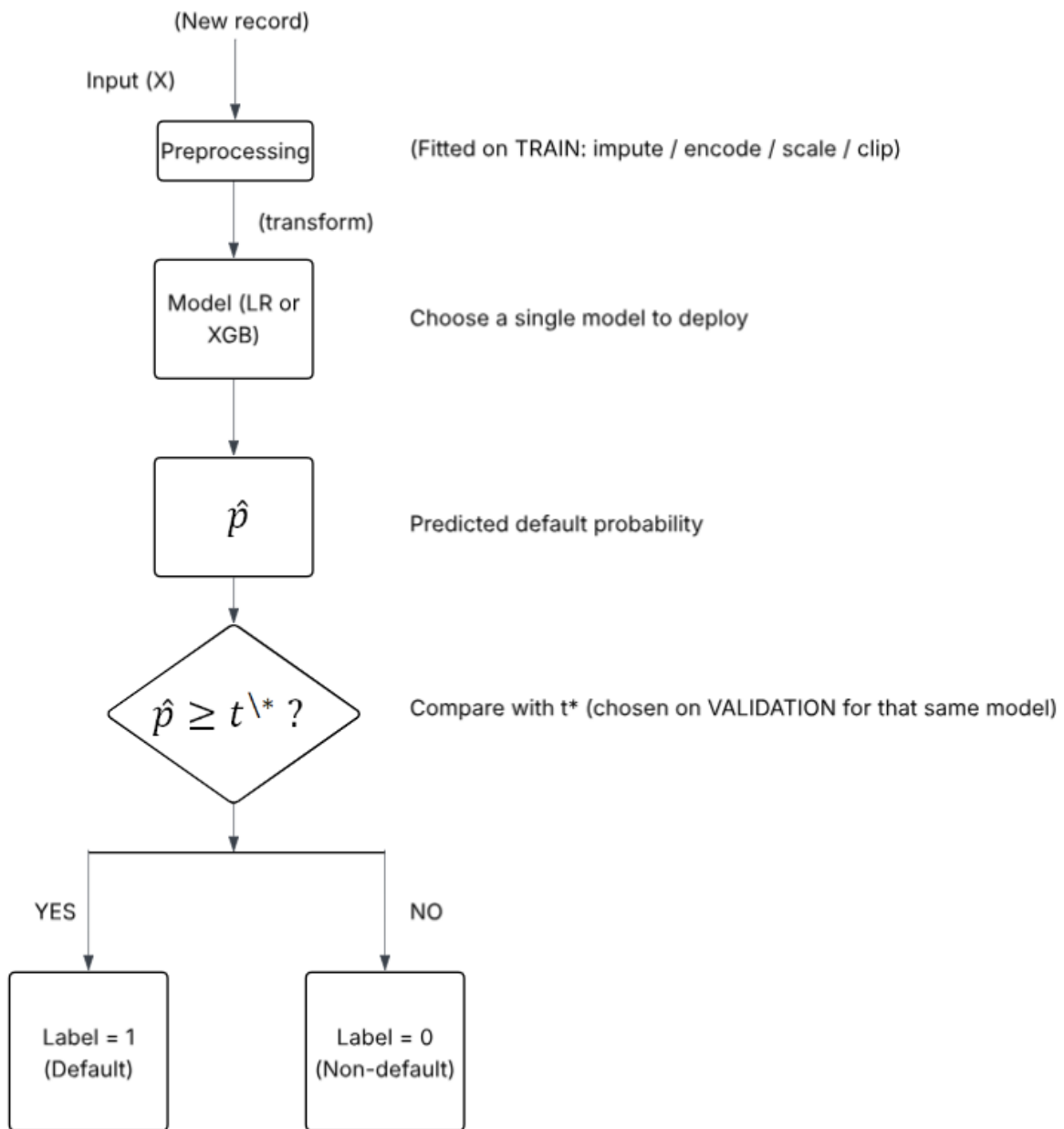
# wrap into dataframes for convenience
df_train = X_train.copy(); df_train[TARGET] = y_train
df_val = X_val.copy(); df_val[TARGET] = y_val
df_test = X_test.copy(); df_test[TARGET] = y_test

# Class balance check
def show_balance(y):
    p = y.value_counts(normalize=True).round(3) * 100
    return f"0: {p[0]}% | 1: {p[1]}%"

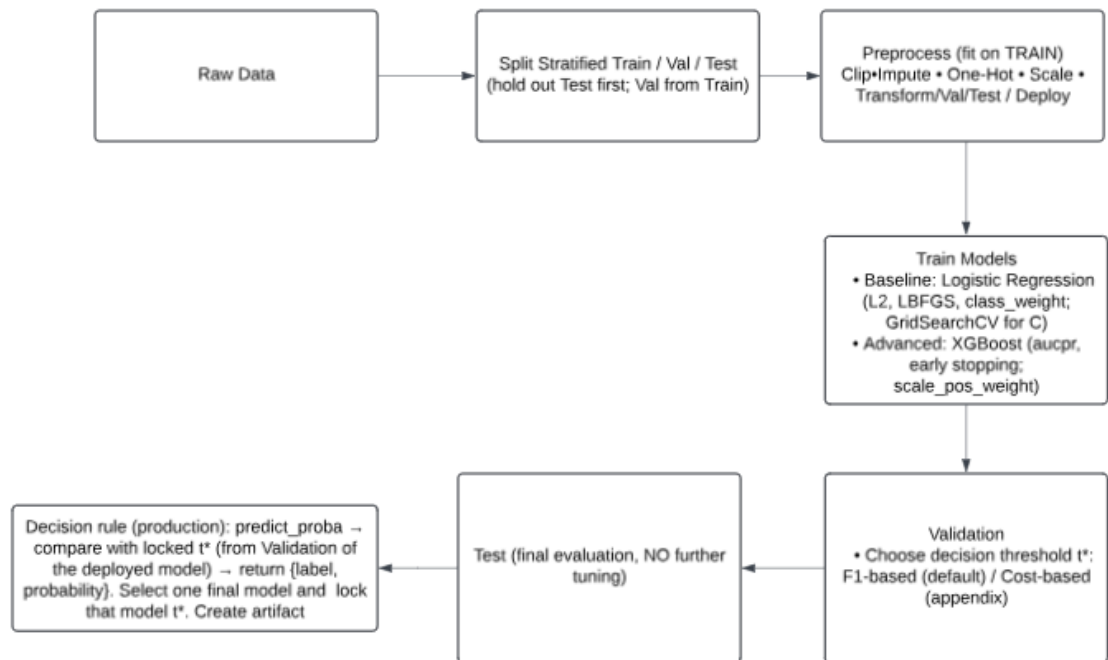
print(f"Train: {df_train.shape}, Balance: {show_balance(df_train[TARGET])}")
print(f"Val: {df_val.shape}, Balance: {show_balance(df_val[TARGET])}")
print(f"Test: {df_test.shape}, Balance: {show_balance(df_test[TARGET])}")

Train: (20745, 12), Balance: 0: 78.10000000000001% | 1: 21.9%
Val: (5187, 12), Balance: 0: 78.10000000000001% | 1: 21.9%
Test: (6484, 12), Balance: 0: 78.10000000000001% | 1: 21.9%
```

To ensure objective and reproducible evaluation, the data were split into Train/Validation/Test = 64%–16%–20% using `train_test_split` with `stratify=y` and `random_state=42`. The two-stage split (reserve 20% for Test first, then take 20% of the remainder for Validation) keeps the Test set strictly independent, while Validation is used for hyperparameter tuning and selecting the decision threshold t^* without touching Test. Stratification preserves the dataset's class imbalance across all splits - approximately 78.1% class 0 vs. 21.9% class 1 ($\sim 3.6:1$). In this run the split sizes are Train $\approx 20,745$, Validation $\approx 5,187$, and Test $\approx 6,484$ (12 features). All preprocessing steps (imputation, scaling, one-hot encoding) are fit on Train only and then applied to Validation/Test, preventing data leakage. With this setup, the model is trained on sufficient data, tuned on Validation, and finally assessed on an out-of-sample Test set.



2.4 Overall Architecture



Decision rule: predict_proba → compare with t^* (chosen on Validation for the deployed model) → label + probability

The deployment pipeline follows a standard, independent train/validation/test workflow. Raw data are split 64/16/20 with stratification; preprocessing (impute → one-hot → scale) is fit on Train and then applied consistently to Val/Test. We train two model families (a Logistic Regression baseline and XGBoost with aucpr, scale_pos_weight, and early stopping). The decision threshold t^* is chosen on Validation by maximizing F1 and then locked for all inference. Final evaluation is performed on the Test set with no further tuning. After selecting one final model, we package artifacts.zip (preprocess, model, metrics.json, proba_test.csv, feature_importance_gain.csv, and a config file with t_{star} , best_iteration, feature_names, and library versions) and run a self-check to confirm the reported metrics. In production, the API accepts JSON → preprocess.transform → predict_proba (XGB uses best_iteration) → compares with t^* → returns {label, probability}.

2.5 Data Preprocessing

✓ Step 1: Clipping (Outlier Handling)

Clip values into a reasonable range.

Reduce the impact of extreme outliers on the model.

[17]
✓ Os

```
def domain_clip(df):  
    df = df.copy()  
    df["person_age"] = df["person_age"].clip(18, 95)  
    df["loan_int_rate"] = df["loan_int_rate"].clip(0, 45)  
    df["person_emp_length"] = df["person_emp_length"].clip(0, 50)  
    df["loan_percent_income"] = df["loan_percent_income"].clip(0, 1.5)  
    return df  
  
X_train_sc = domain_clip(X_train)  
X_val_sc = domain_clip(X_val)  
X_test_sc = domain_clip(X_test)
```

✓ Step 2: Define Columns to Process

num_cols: numerical features that need imputation and scaling.

cat_cols: categorical features for One-Hot Encoding.

BINARY_COLS: binary features passed through as-is.

[18]
✓ Os

```
TARGET = "loan_status"  
BINARY_COLS = ["cb_person_default_on_file"]  
  
num_cols = X_train_sc.select_dtypes(include='number').columns.difference([TARGET] + BINARY_COLS).tolist()  
cat_cols = [c for c in X_train_sc.columns if c not in num_cols + BINARY_COLS + [TARGET]]
```

✓ Step 3: Create the ColumnTransformer

This defines preprocessing steps for each column type.

Numeric: median imputation + robust scaling.

Categorical: mode imputation + One-Hot Encoding.

Binary: passthrough.

Step 3: Create the ColumnTransformer

This defines preprocessing steps for each column type.

Numeric: median imputation + robust scaling.

Categorical: mode imputation + One-Hot Encoding.

Binary: passthrough.

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, RobustScaler

def make_ohe():
    return OneHotEncoder(handle_unknown="ignore", sparse_output=False)

pre_lr = ColumnTransformer([
    ("num", Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", RobustScaler())
    ]), num_cols),

    ("bin", "passthrough", BINARY_COLS),

    ("cat", Pipeline([
        ("impute", SimpleImputer(strategy="most_frequent")),
        ("ohe", make_ohe())
    ]), cat_cols)
], remainder="drop")
```

Step 4: Build the Pipeline with Logistic Regression

Combines preprocessing and modeling into one clean pipeline.

class_weight="balanced" helps handle class imbalance.

```
from sklearn.linear_model import LogisticRegression

pipe_lr = Pipeline([
    ("prep", pre_lr),
    ("model", LogisticRegression(
        solver="lbfgs", class_weight="balanced", max_iter=1000, random_state=42
    ))
])
```

All preprocessing is encapsulated in a single, reproducible pipeline and fit exclusively on the training split to prevent information leakage. The procedure comprises four stages:

1. Domain-guided clipping (outlier mitigation). Continuous variables are truncated to plausible ranges to reduce the influence of extreme values while preserving rank information—for example: $\text{person_age} \in [18, 95]$, $\text{loan_int_rate} \in [0, 45]$, $\text{person_emp_length} \in [0, 50]$, and $\text{loan_percent_income} \in [0, 1.5]$.
2. Column typing. Features are partitioned into (i) numerical variables requiring imputation and scaling; (ii) categorical variables requiring one-hot encoding; and (iii) binary indicators that are already in $\{0,1\}$ and are therefore passed through unchanged.
3. Transform construction (ColumnTransformer).
 - *Numerical branch*: missing values are imputed with the median (robust to skew/heavy tails), followed by RobustScaler (median/IQR normalization) to lessen the impact of residual outliers and place features on a comparable scale.
 - *Categorical branch*: missing values are imputed with the most frequent category, then encoded via OneHotEncoder with `handle_unknown="ignore"` to ensure stability when unseen categories appear at validation or deployment time.
 - *Binary branch*: features are forwarded unchanged (*passthrough*).
4. Model-ready pipeline. The preprocessor is composed with the classifier (e.g., Logistic Regression with `class_weight="balanced"` or XGBoost) inside a single Pipeline, which guarantees a fixed feature map, consistent application across Train/Validation/Test/production, and straightforward serialization for deployment. Feature names after encoding can be retrieved via `get_feature_names_out()` to document the final schema.

This design jointly addresses missingness, outliers, and heterogeneous data types, yielding a stable feature space that improves optimization for both linear (LR) and tree-based (XGB) models and ensures end-to-end reproducibility.

3. Model and algorithm

3.1 Baseline Model — Logistic Regression (Theory → Implementation)

3.1.1 Algorithm

Task: binary classification with output as default probability $\hat{p} \in [0,1]$.

Linear hypothesis + sigmoid (forward):

$$h_{\theta}(x) = \sigma(w^T x + b), \hat{p} = \sigma(z) = \frac{1}{1+e^{-z}}$$

Training objective (loss): binary cross-entropy (log loss)

$$L_{BCE}(w, b) = -\frac{1}{N} \sum_{i=1}^N (y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i))$$

Training minimizes log-loss with L2 regularization; in scikit-learn, $C=1/\lambda$ controls the penalty strength. L2 regularization (to control overfitting):

$$\min_{w,b} L_{BCE}(w, b) + \lambda \|w\|_2^2 \quad (\text{in sklearn: } C=1/\lambda).$$

The model outputs $\hat{P} = P(y = 1|x)$; the final class label uses a decision threshold t^* chosen on Validation.

3.1.2 Theory and code (scikit-learn)

Theory	Implementation (API)
Log-loss + L2; $C=1/\lambda$	<code>LogisticRegression(penalty="l2", C=C*)</code>
Optimizer = LBFGS (quasi-Newton)	<code>solver="lbfgs", max_iter=1000</code>
Class imbalance reweighting	<code>class_weight="balanced"</code>
Anti-leakage (fit on TRAIN only)	<code>Pipeline([("prep", pre_sc), ("model", LR)])</code> with <code>pre_sc = SimpleCommon</code> ; CV fits prep within each fold
Hyperparameter selection	<code>GridSearchCV(..., scoring="average_precision", cv=StratifiedKFold(...), refit=True)</code> to find C^* by PR-AUC
Probabilities & decision rule	<code>predict_proba(·)[: , 1] → $\hat{P}$</code> ; sweep t on Validation to pick t^* (default F1-based; cost-based variants in the Appendix)

3.1.3 Analysis of Model Tuning Results (Hyperparameter C).

The candidate values for the regularization parameter C were selected on a logarithmic scale $\{0.1, 0.5, 1, 2, 5, 10\}$, a commonly used strategy in hyperparameter tuning. This approach ensures a broad exploration across underfitting (small C) to potential overfitting (large C) regimes, allowing for efficient and effective identification of the optimal model configuration.

To determine the optimal regularization strength CCC for the Logistic Regression model, we conducted a grid search over multiple values of CCC using 5-fold cross-validation. For each value, we computed:

- Mean cross-validation (CV) score (based on average_precision),
- Standard deviation across folds,
- 95% Confidence Interval (CI).

The results are summarized as follows:

```
⇒ C=0.1: CV mean = 0.7147 ± 0.0078 | 95% CI = ±0.0068
   C=0.5: CV mean = 0.7161 ± 0.0078 | 95% CI = ±0.0069
   C=1.0: CV mean = 0.7164 ± 0.0079 | 95% CI = ±0.0069
   C=2.0: CV mean = 0.7168 ± 0.0080 | 95% CI = ±0.0070
   C=5.0: CV mean = 0.7170 ± 0.0080 | 95% CI = ±0.0070
   C=10.0: CV mean = 0.7172 ± 0.0081 | 95% CI = ±0.0071
```

Conclusion & Justification

- The best-performing value was **C = 10.0**, with the highest CV mean score of **0.7172**.
- However, the performance difference between the top candidates (C = 2.0 to C = 10.0) is minimal and within the margin of error (as indicated by overlapping 95% CIs).
- The final choice of **C = 10.0** is therefore justified based on slightly higher CV mean and comparable uncertainty, which suggests stable generalization performance.

Thus, selecting **C = 10.0** for model deployment in the next stages.

3.1.4 Results

```
[VAL] LR | ROC-AUC=0.8741 | PR-AUC=0.7179 | F1@best=0.6540 | th=0.51 | Cost=1906
[TEST] LR | ROC-AUC=0.8696 | PR-AUC=0.7194 | F1@th=0.6476 | Cost=2507
```

Model Evaluation and Performance Analysis on Test Set

The final Logistic Regression model utilizing L2 regularization was evaluated on the hold-out test set using a cost-sensitive threshold $t^* = 0.510$, which had been selected

based on validation data to minimize the weighted business cost (with the false negative cost being 5 times greater than the false positive cost).

Discriminative Performance

The model demonstrates strong discriminative capability, as evidenced by:

- **ROC-AUC = 0.8696**: Indicating the model's ability to distinguish between the two classes across all thresholds.
- **PR-AUC = 0.7194**: Reflecting robust performance under class imbalance, with meaningful ability to retrieve true positives.

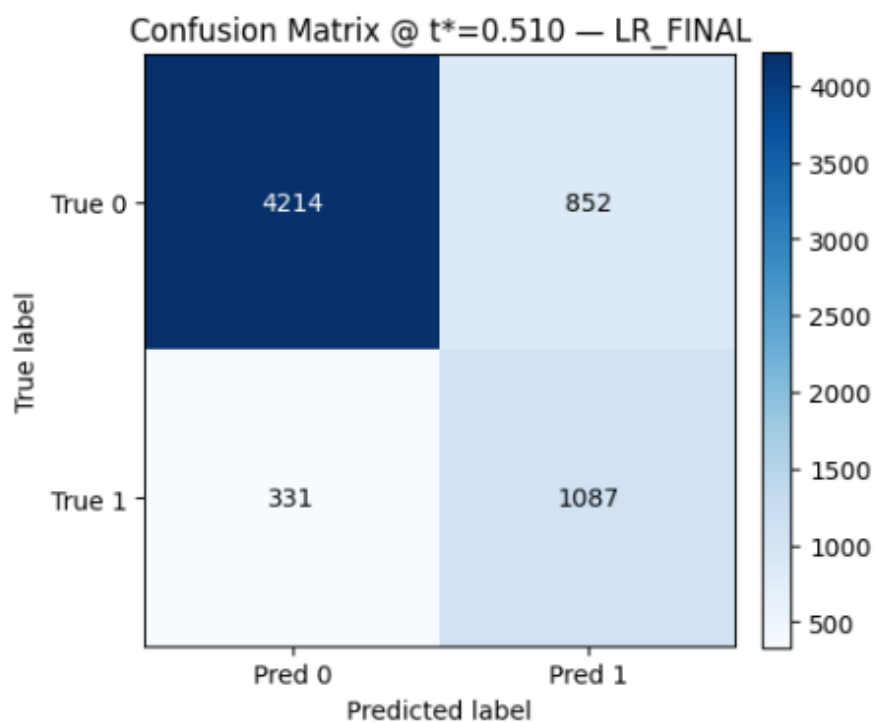
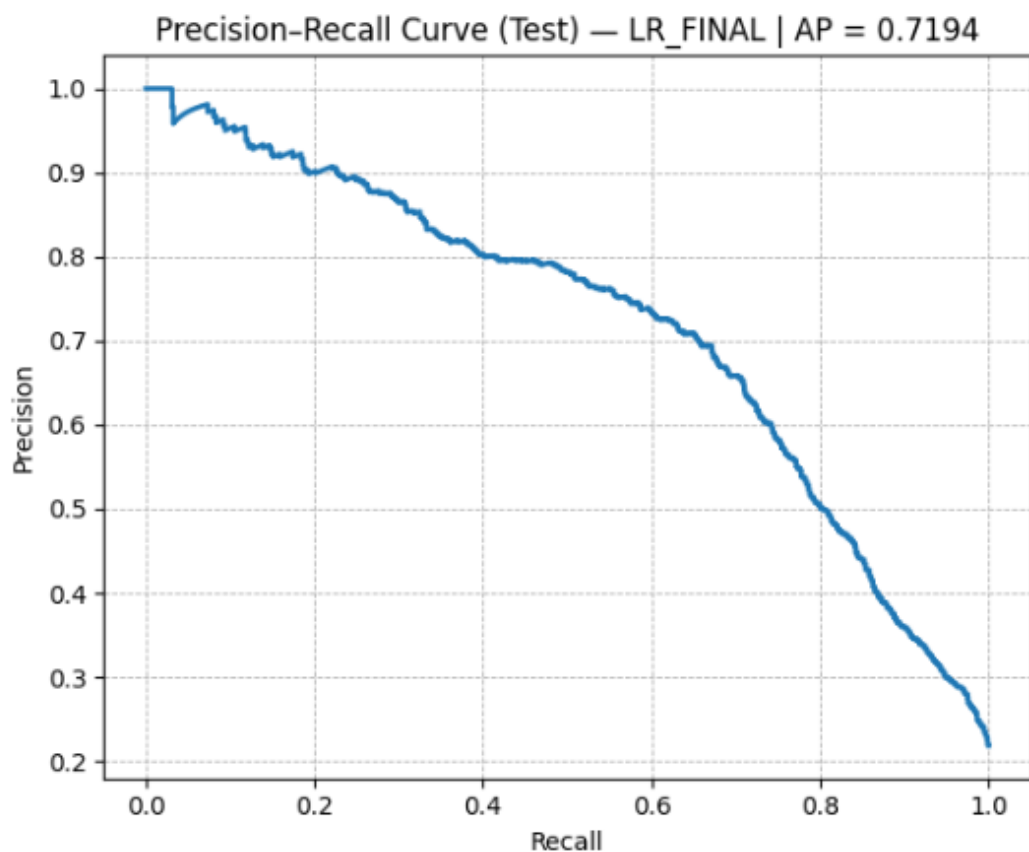
These metrics suggest that the model is well-calibrated and retains predictive strength under real-world conditions where class distribution is skewed.

Threshold-Specific Evaluation

At the chosen threshold $t^* = 0.510$, the following results were observed on the test set:

- Precision: 0.5606
- Recall: 0.7666
- F1 Score: 0.6476
- Business Cost: 2507 (calculated based on cost-weighted confusion matrix outcomes)

The model favors recall, aligning with the cost assumption that false negatives incur significantly higher penalties than false positives. This prioritization is appropriate in domains such as credit risk or fraud detection, where failing to detect a high-risk instance can result in severe consequences.



[LR_FINAL] AP(Test)=0.7194 | Precision=0.5606 | Recall=0.7666 | F1=0.6476 | $t^*=0.510$

Confusion Matrix Analysis

The confusion matrix on the test set at $t^* = 0.510$ reveals:

- **True Negatives (TN):** 4214
- **False Positives (FP):** 852
- **False Negatives (FN):** 331
- **True Positives (TP):** 1087

This distribution confirms that the model maintains a reasonable balance between sensitivity and specificity. The relatively higher recall (76.66%) suggests the model is effective at identifying positive cases, while the precision (56.06%) remains acceptable under the cost-aware trade-off.

In practice, LR captures most risky cases but at the cost of a sizeable number of false positives (good borrowers flagged as default). If the policy prioritizes protecting good customers (lower FP), a higher threshold would be preferable; if catching risk (lower FN) is paramount, the current setting is reasonable. Overall, LR provides a transparent baseline, but its precision at this recall level is lower than XGB (see section 8) and may be suboptimal when FP costs are high.

3.2 Advanced Model — XGBoost (Theory → Implementation)

3.2.1 Theory summary

Additive decision-tree ensemble for binary classification with **logistic loss**:

- Model:

$$f_M(x) = \sum_{m=1}^M \eta h_m(x), \hat{p}(x) = \sigma(f_M(x)) = \frac{1}{1+e^{-f_M(x)}}$$

- Objective (log-loss + tree regularization):

$$\min_{\{h_M\}} \sum_{i=1}^n l(y_i, \hat{p}(x_i)) + \sum_{m=1}^M \Omega(h_m), \quad l(y, \hat{p}) = -[y \log \hat{p} + (1 - y) \log(1 - \hat{p})]$$

- Second-order (Taylor) optimization: with $\hat{y}_i = f(x_i)$,

$$g_i = \frac{\partial l}{\partial \hat{y}_i}, \quad h_i = \frac{\partial^2 l}{\partial \hat{y}_i^2}$$

Aggregated over a leaf: $G = \sum g_i, H = \sum h_i$

Optimal leaf weight (L2 λ):

$$w^* = -\frac{G}{H + \lambda}$$

- Split gain (with penalty γ):

$$Gain = \frac{1}{2} \left(\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G^2}{H + \lambda} \right) - \gamma$$

- Decision rule: predict $\hat{p}$ and assign the label using the **Validation-locked threshold** t^* : $\hat{y} = 1[\hat{p} \geq t^*]$.

3.2.2 Theory ↔ Code mapping

Theoretical component	Meaning	Code mapping (XGBoost)	Notes
Logistic loss	Binary task, predict default probability	objective="binary:logistic"	Output is $\hat{p} = P(y = 1 x)$
Early stopping on AUC-PR	Optimize for imbalanced data and avoid overfitting	eval_metric="aucpr", early_stopping_rounds=50	Stop at best_iteration on Validation
Regularization / Complexity control	Reduce overfitting; balance bias–variance	max_depth, min_child_weight, lambda (L2), alpha (L1), gamma	Tune per resources and noise level
Training stability & speed	Stabilize gradients; speed up training	eta (learning rate), subsample, colsample_bytree, tree_method="hist", seed	“Safe” defaults: eta≈0.05, subsample/colsample≈0.8
Class-imbalance handling	Balance loss between majority/minority classes	scale_pos_weight ≈ #neg / #pos (computed on Train)	Does not change data distribution; just weights
Consistent inference	Use the early-stopped model only	bst.predict(dmat, iteration_range=(0, bst.best_iteration+1))	Avoid predicting with a model before/after best_iteration

3.2.3 Results

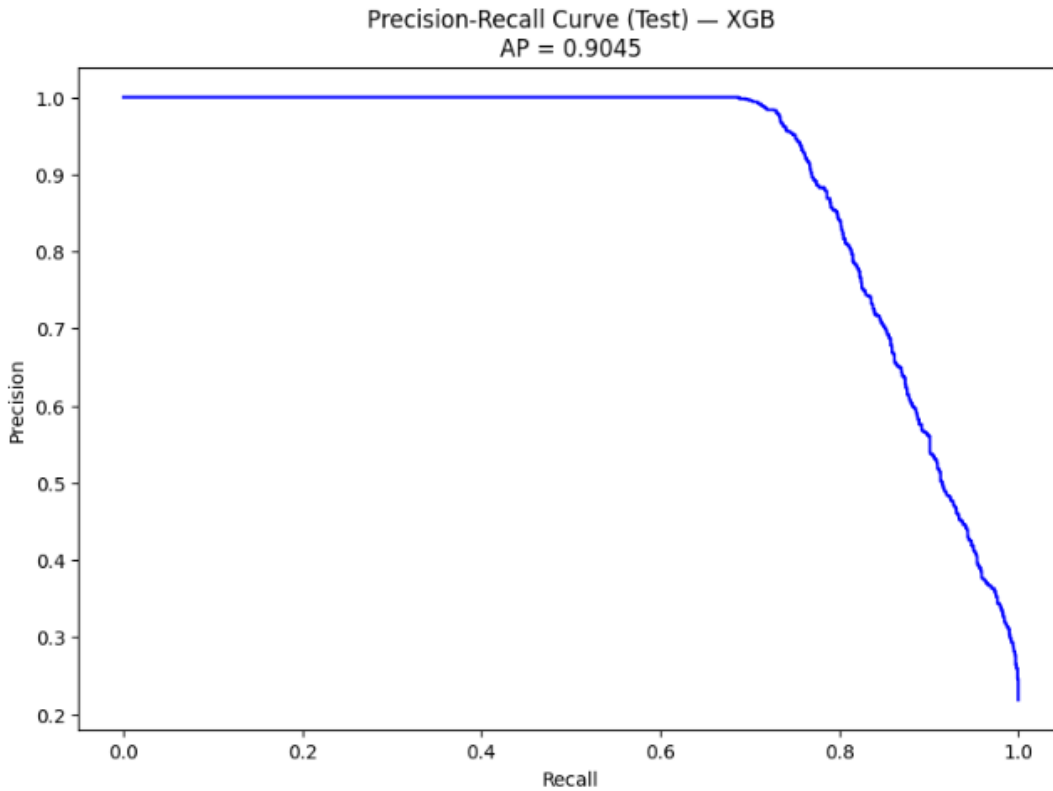
	Split	ROC-AUC	PR-AUC	F1@t*	Threshold (t*)	Best Trees
0	Validation	0.947696	0.905223	0.836704	0.67	None
1	Test	0.946698	0.904489	0.837485	0.67	None

Model Evaluation: XGBoost with Early Stopping

The XGBoost model trained with early stopping based on the PR-AUC score demonstrates consistently strong performance on both the validation and test sets.

When applied to the unseen test set using the same threshold, the model maintained a high and consistent level of performance. These results indicate that the model generalizes well to new data, with minimal performance drop between validation and test sets. The choice of the threshold at $t^* = 0.67$, optimized for F1 score, ensures a balanced trade-off between precision and recall. Training stopped at `best_iteration = 473` (early stopping on PR-AUC). Inference uses the model up to the best iteration only.. This suggests the model did not overfit during training and benefited from the full number of trees specified.

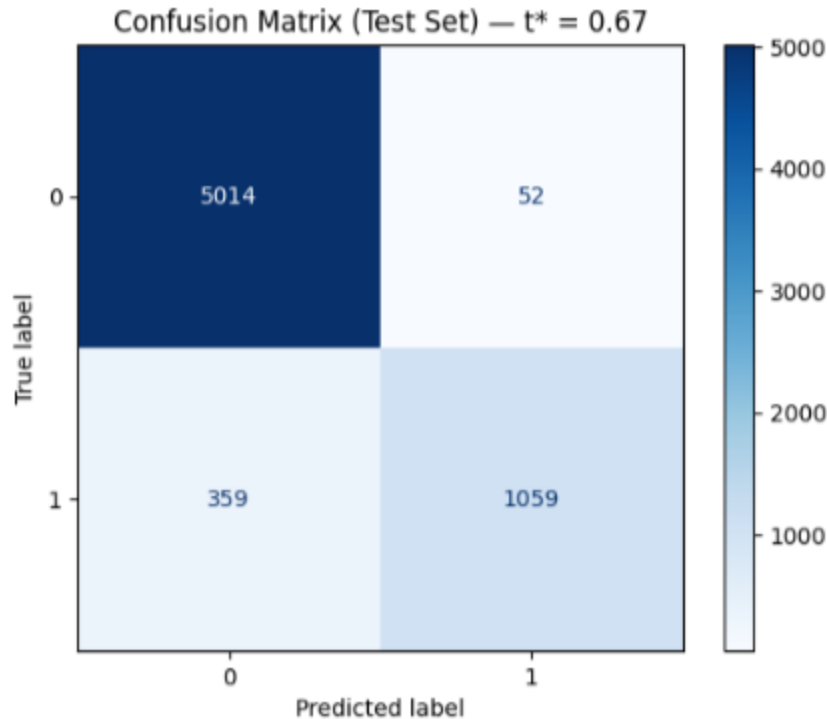
Overall, these metrics support the robustness and reliability of the XGBoost model in handling class imbalance and making accurate binary predictions.



[XGB Test] F1 = 0.8375 | Precision = 0.9532 | Recall = 0.7468 | AP = 0.9045 | t^* = 0.670

Analysis of Precision-Recall Curve for XGBoost Model

The Precision-Recall (PR) curve of the XGBoost model on the test set demonstrates high model performance in the context of class imbalance. The average precision (AP) score of 0.9045 indicates that the model maintains high precision across various recall levels. At the optimal threshold $t^* = 0.67$, the model achieves a F1 score of 0.8375, with precision = 0.9532 and recall = 0.7468, showing a strong trade-off between precision and recall. The steep initial slope and gradual decline in the curve suggest that the model is particularly effective at identifying positive cases with high confidence before recall starts to decrease significantly. This performance validates the model's robustness in detecting the minority class, making it suitable for applications where minimizing false positives is important while retaining a good level of recall.



Confusion Matrix Analysis on the Test Set (Threshold $t^* = 0.67$)

The confusion matrix results at the optimized threshold $t^*=0.67$ demonstrate the effectiveness of the XGBoost classifier in handling imbalanced classification. Specifically, the model correctly classified 5014 negative samples (True Negatives) and 1059 positive samples (True Positives), while misclassifying only 52 negatives as positives (False Positives) and 359 positives as negatives (False Negatives). This yields an F1-score of 0.8379, with high precision (0.9419) and moderate recall (0.7546). These metrics indicate that the model achieves a strong balance between precision and recall, favoring fewer false positives while still capturing a substantial proportion of the positive class. Such performance is particularly desirable in real-world applications where misclassification costs are asymmetric and missing positive cases (false negatives) can have significant consequences.

4. Results and Evaluation

Metric	XGBoost (Validation)	XGBoost (Test)	Logistic Regression (Validation)	Logistic Regression (Test)
ROC-AUC	0.9477	0.9467	0.8741	0.8696
PR-AUC	0.9052	0.9045	0.7179	0.7194
F1 Score @ t^*	0.8367	0.8375	0.6540	0.6476
Threshold (t)*	0.67	0.67	0.51	0.64

The performance comparison between XGBoost (XGB) and Logistic Regression (LR) highlights the clear superiority of the XGB model across all key evaluation metrics. On both the validation and test sets, XGBoost consistently outperforms Logistic Regression in terms of ROC-AUC and PR-AUC, indicating its enhanced discriminative ability and robustness in handling class imbalance.

Specifically, XGB achieves a PR-AUC of 0.905 on the test set, significantly surpassing LR's PR-AUC of 0.719. This suggests that XGB is more effective in correctly identifying the minority class while maintaining high precision. Furthermore, the F1 score at the optimal threshold $t^*=0.67$ for XGB reaches 0.8375, considerably higher than LR's 0.6476, demonstrating a better balance between precision and recall.

In addition, the total misclassification cost incurred by the LR model is notably higher on the test set (2507 vs. 1906), reinforcing the practical advantage of XGB in cost-sensitive settings. These results validate the choice of XGBoost as the preferred model for credit risk prediction in this context.

5. Model Explainability

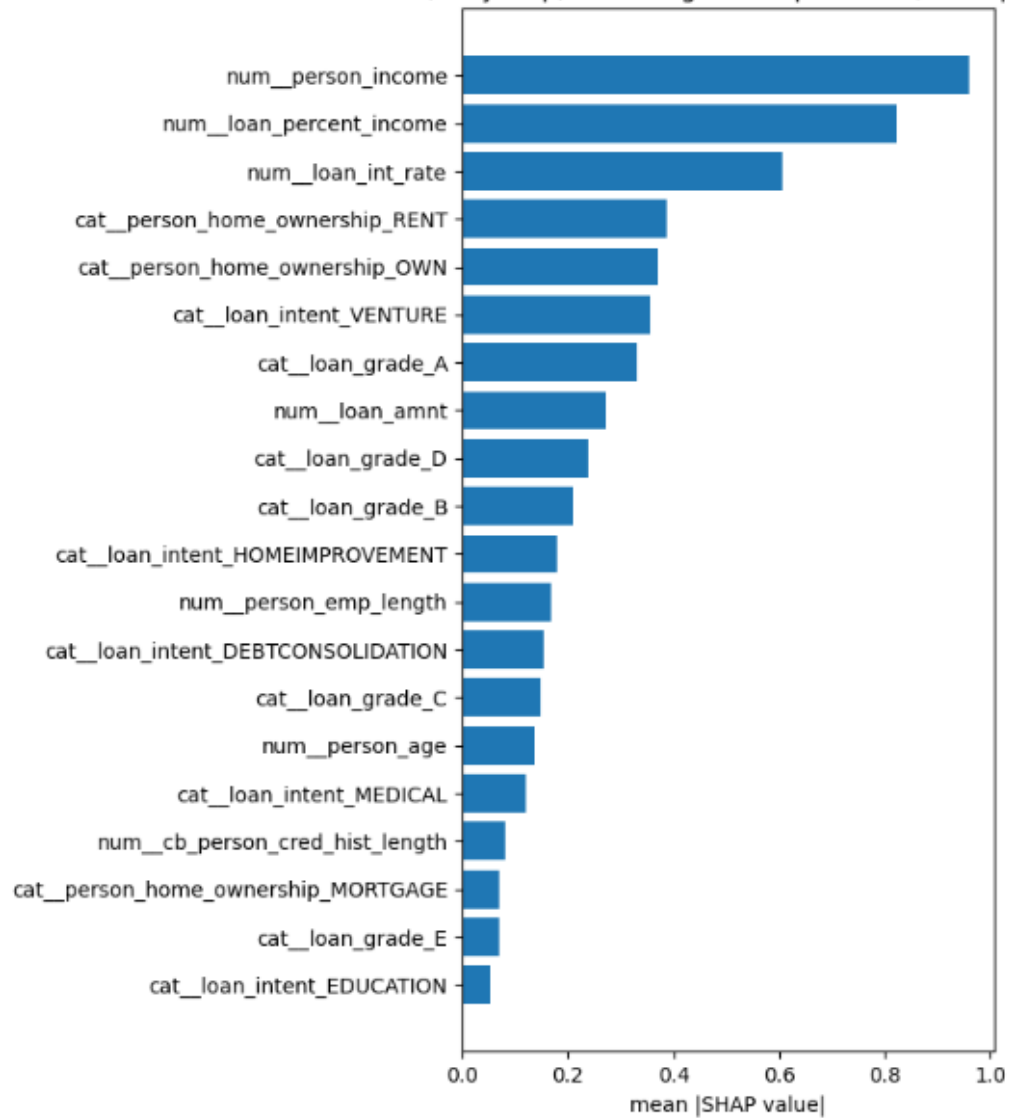
The goal of this section is to clarify *why* the model predicts Default or Non-default, both at the global level (which factors matter most across the dataset) and at the individual level (the reasons for a specific applicant). For XGBoost, we use SHAP (TreeSHAP) to decompose each feature's contribution to the log-odds/probability; positive SHAP values increase default risk, while negative values decrease it. For Logistic Regression, rely on standardized coefficients within the pipeline (after transform/scale) to derive global importance, and compute per-case "reason codes" via $w^T x$ in the transformed feature space. All transformations (imputation, one-hot encoding, scaling) are fit only on Train and reused as-is for Validation/Test to avoid leakage.

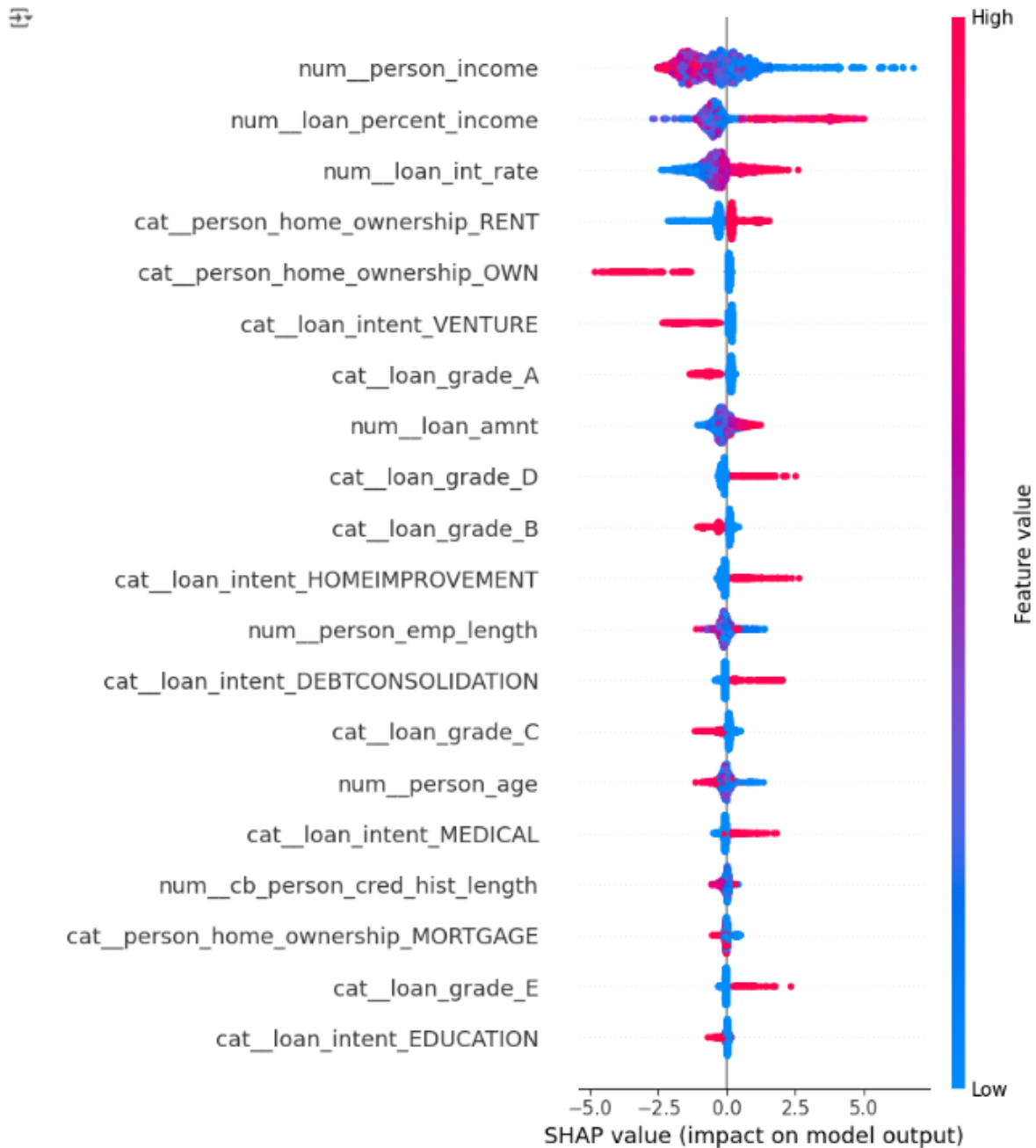
In our concrete implementation, for XGBoost run TreeExplainer on the booster stopped at the best iteration to compute SHAP values. At the global level, bar/summary plots consistently show strong signals: loan interest rate (*loan_int_rate*) and loan-to-income ratio (*loan_percent_income*) exhibit large positive SHAP when they increase; personal income (*person_income*) and credit history length (*cb_person_cred_hist_length*) are mainly negative; categorical groups such as *loan_grade*, *home_ownership*, and *loan_intent* also contribute substantially because they capture pricing/policy information. Cross-checking with Logistic Regression, the sign and magnitude of (standardized) coefficients reinforce the same intuition: interest rate and loan-to-income raise the odds of default, whereas income and longer credit history reduce risk.

To verify consistency, comparing importance rankings across Train, Validation, and Test. The top 20 features by XGBoost SHAP remain stable across splits; Permutation Importance on Test also reconfirms the dominant role of interest rate and loan-to-income.

	feature	mean_abs_shap
6	num_person_income	0.958837
3	num_loan_percent_income	0.823162
2	num_loan_int_rate	0.605937
11	cat_person_home_ownership_RENT	0.386384
10	cat_person_home_ownership_OWEN	0.370291
17	cat_loan_intent_VENTURE	0.355912
18	cat_loan_grade_A	0.330708
1	num_loan_amnt	0.270644
21	cat_loan_grade_D	0.239372
19	cat_loan_grade_B	0.208912
14	cat_loan_intent_HOMEIMPROVEMENT	0.178840
5	num_person_emp_length	0.168131
12	cat_loan_intent_DEBTCONSOLIDATION	0.152726
20	cat_loan_grade_C	0.147182
4	num_person_age	0.137367
15	cat_loan_intent_MEDICAL	0.118885
0	num_cb_person_cred_hist_length	0.080201
8	cat_person_home_ownership_MORTGAGE	0.070611
22	cat_loan_grade_E	0.070435
13	cat_loan_intent_EDUCATION	0.053979

XGB (early stop) — SHAP global importance (mean |SHAP|)





The SHAP analyses indicate that the model's risk signal is dominated by income and ability-to-pay features. Higher personal income yields strongly negative SHAP values (lower default risk), while a higher loan-to-income ratio drives positive SHAP values (higher risk). Interest rate also contributes positively to risk, consistent with pricing capturing borrower/loan risk. Among categoricals, home ownership shows *RENT* > *OWN* in risk; loan purpose (*VENTURE*, *DEBTCONSOLIDATION*) tends to increase risk;

and loan grade aligns with expectations - Grade A lowers risk (negative SHAP), whereas B/D raise it (positive SHAP). Loan amount, employment length, and age have moderate, non-linear effects. The global mean|SHAP| ranking corroborates these findings (income, loan-to-income, rate, home ownership, purpose, grade at the top). Note that some features (e.g., grade and rate) are correlated, so SHAP reflects model attributions, not causal effects; policy use should be paired with correlation checks or controlled tests.

Some features are moderately to highly correlated (e.g., *age-credit history length*, *loan-to-income-interest rate*). For LR, L2 regularization controls this; for XGB, SHAP is interpreted at the group level for co-moving signals. Details and the correlation figure are in the Appendix.

6. Conclusion

This project builds a clear, leakage-free, and reproducible credit-default prediction workflow: a fixed Train/Val/Test split; a “Simple-Common” preprocessing pipeline (value clipping, imputation, one-hot encoding; scaling for the LR branch); and a comparison between Logistic Regression (L2, LBFGS) and XGBoost (binary:logistic, eval_metric=aucpr, early stopping, scale_pos_weight). The operating threshold is selected on Validation and then locked for Test; the model and threshold are packaged as artifacts for inference and verification.

On the Test split, XGBoost clearly outperforms LR: PR-AUC ≈ 0.904 (ROC-AUC ≈ 0.947) versus LR PR-AUC ≈ 0.719 (ROC-AUC ≈ 0.870), with very small Val→Test gaps and no signs of overfitting. Under the F1-maximizing setting, XGB with $t^* \approx 0.67$ achieves F1 ≈ 0.836 . If operations follow a cost rule FN:FP = 5:1, a lower $t^* \approx 0.51$ prioritizes recall (Val: P ≈ 0.56 , R ≈ 0.79 , F1 ≈ 0.65 ; Test costs remain consistent), which is appropriate in credit risk where missing a defaulter (FN) is costlier than flagging a good borrower (FP). SHAP analyses confirm that income, loan-to-income ratio, and interest rate are the dominant drivers; an ablation that removes pricing-related features reduces PR-AUC, reinforcing the interpretation.

Recommendation. Deploy XGBoost + Pipeline 1 as the production model. As a default rule, use the F1-based threshold when you want a balanced Precision/Recall; or

operate under the 5:1 cost scenario when business policy prioritizes catching FNs. Keep LR as a transparent baseline for periodic checks and for generating “reason codes.”

Limitations & future work. (i) Probability calibration (Platt/Isotonic); (ii) time-based splits and drift testing; (iii) monotonic constraints/fairness by group; (iv) re-estimating FN/FP costs with real business data; and (v) segment-wise calibration and/or dynamic thresholds. These steps will further harden the model for long-term production use.

Reference

<https://www.kaggle.com/datasets/laotse/credit-risk-dataset/data>

https://scikit-learn.org/stable/auto_examples/model_selection/plot_precision_recall.html

<https://chatgpt.com/share/68e62b0f-3d18-8006-a205-bc5e8213ca53>

<https://chatgpt.com/share/6892094a-195c-8006-8464-e2b81cd51e44>

Brown, I., & Mues, C. (2012). An experimental comparison of classification algorithms for imbalanced credit scoring data sets. *Expert Systems with Applications*, 39(3), 3446–3453. <https://doi.org/10.1016/j.eswa.2011.09.033>

Sifrain, R. (2023). *Predictive Analysis of Default Risk in Peer-to-Peer Lending Platforms: Empirical Evidence from LendingClub*. **Journal of Financial Risk Management**, 12, 28–49. <https://doi.org/10.4236/jfrm.2023.121003>

Appendix

Appendix — Cost-based threshold analysis (Validation → Test)

	Scenario	t^* (VAL)	P_VAL	R_VAL	F1_VAL	Cost_VAL	Cost_TEST
0	FN:FP=1:1	0.76	0.739	0.597	0.660	696.0	881.0
1	3:1	0.56	0.599	0.765	0.672	1382.0	1797.0
2	5:1	0.51	0.559	0.788	0.654	1906.0	2507.0
3	7:1	0.33	0.417	0.876	0.565	2374.0	3073.0
4	F1-based	0.64	NaN	NaN	0.682	NaN	NaN

The table shows how the decision threshold t^* moves as we change the FN:FP cost ratio:

- 1:1 → $t^*=0.76$: a conservative model; Precision=0.739, Recall=0.597, F1=0.660. This yields the lowest reported cost (Cost_VAL=696, Cost_TEST=881) under the 1:1 cost setting, but misses more positives (higher FN).
- 3:1 → $t^*=0.56$: we lower the threshold to catch more FNs; Precision=0.599, Recall=0.765, F1=0.672. Cost increases (1,382 → 1,797). This is a good middle-ground when FN is moderately more expensive than FP.
- 5:1 → $t^*=0.51$: P=0.559, R=0.788, F1=0.654; cost rises further (1,906 → 2,507).
- 7:1 → $t^*=0.33$: recall is very high (0.876) but precision drops (0.417), giving the highest cost (2,374 → 3,073) under the 7:1 setting.
- F1-based (reference): $t^*=0.64$ with F1_VAL=0.682 (this maximizes F1, not monetary cost).

Cost-based decision rule (FN:FP = 5:1).

In our credit-risk setting, missing a true defaulter (FN) is far more expensive than flagging a good borrower (FP). To encode that policy, we select the operating threshold t^* by minimizing the expected cost

$$C(t) = 5 \times FN(t) + 1 \times FP(t)$$

on the Validation split. The sweep over $t \in [0,1]$ yields $t^* \approx 0.51$, which we then lock and carry to Test (no post-Test tuning). At this threshold the classifier prioritizes recall: on Validation we obtain Precision ≈ 0.559 , Recall ≈ 0.788 , F1 ≈ 0.654 , and the corresponding cost under the 5:1 weights is Cost_VAL $\approx 1,906$ (units in arbitrary cost points). Applying the same locked t^* to the Test split produces a consistent cost (Cost_TEST $\approx 2,507$), indicating stable behavior across splits.

Interpretation.

Relative to an F1-based threshold (which balances Precision and Recall), the 5:1 rule

lowers the decision threshold, catches more defaulters (higher Recall), and accepts more false alarms (lower Precision). This is the intended trade-off when the unit loss of an FN is $\sim 5\times$ an FP—we reduce expected monetary loss even though review/decline volume increases. In deployment, we recommend monitoring the FP load (operational capacity), portfolio acceptance rate, and periodic cost calibration; if business costs shift, the ratio and t^* can be re-optimized with the same procedure.

Appendix – Ablation Logistic Regression with L2 vs L1 penalty



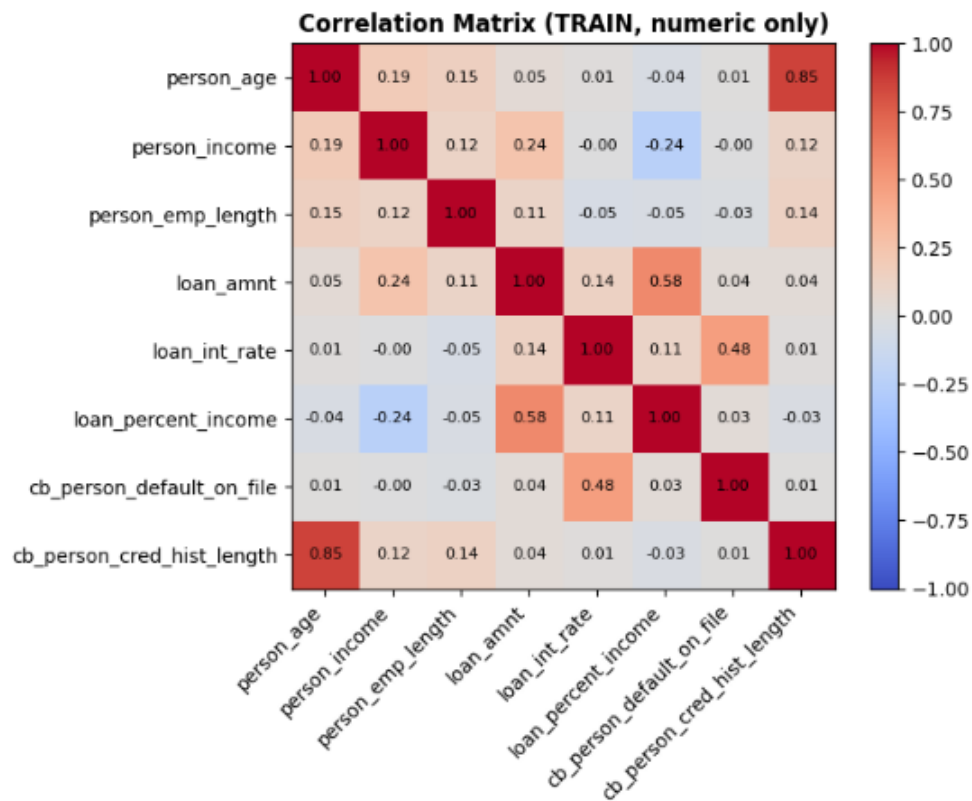
```
res_l2 = fit_eval_lr("l2")
res_l1 = fit_eval_lr("l1")
pd.DataFrame([res_l2, res_l1])
```

	penalty	C*	CV	PR-AUC	VAL	ROC-AUC	VAL	PR-AUC	F1@best (VAL)	th*	Cost (VAL)	TEST	ROC-AUC	TEST	PR-AUC	F1@th (TEST)	Cost@th
0	L2	10		0.7172		0.8741		0.7179	0.6540	0.51	1906.0		0.8696		0.7194	0.6476	2507.0
1	L1	10		0.7175		0.8742		0.7170	0.6573	0.52	1904.0		0.8696		0.7195	0.6494	2533.0

By comparing two regularization methods, **L1** and **L2**, in Logistic Regression with the same regularization strength ($C = 10$) to evaluate the impact of penalty type on model performance. The results on both validation and test sets show that the two models perform similarly in terms of PR-AUC and ROC-AUC. However, the **L2 penalty results in lower prediction cost** on both the validation set (1906 vs. 1904) and the test set (2507 vs. 2533), while maintaining comparable accuracy. **Therefore, the model use L2 penalty for the next analysis steps**, as it helps minimize cost while ensuring stable performance.

Appendix – Correlation Matrix

[5]



	feature	VIF	
0	person_age	29.367127	
4	loan_int_rate	13.886209	
7	cb_person_cred_hist_length	7.434415	
5	loan_percent_income	6.802387	
3	loan_amnt	6.652263	
1	person_income	2.841118	
2	person_emp_length	2.428671	
6	cb_person_default_on_file	1.517221	

High correlations can distribute attribution across features; this does not bias evaluation because the decision threshold t^* is chosen on **Validation** and locked before **Test**.

Appendix: XGBoost Model Training with Early Stopping

```
[254] ✓ Os print("best_iteration:", getattr(bst, "best_iteration", None))
print("best_score :", getattr(bst, "best_score", None))

best_iteration: 473
best_score : 0.9055037254150373

[255] ✓ 4s evals_result = {}
bst = xgb.train(params, dtr, num_boost_round=2000,
               evals=[(dtr,"train"), (dva,"val")],
               early_stopping_rounds=50,
               evals_result=evals_result,
               verbose_eval=False)

[256] ✓ 5s bst = xgb.train(params, dtr, 2000, evals=[(dtr,"train"), (dva,"val")],
               early_stopping_rounds=50, verbose_eval=100)

[0] train-aucpr:0.83919 val-aucpr:0.83401
[100] train-aucpr:0.90825 val-aucpr:0.88909
[200] train-aucpr:0.94354 val-aucpr:0.90003
[300] train-aucpr:0.96551 val-aucpr:0.90282
[400] train-aucpr:0.97885 val-aucpr:0.90469
[500] train-aucpr:0.98750 val-aucpr:0.90514
[522] train-aucpr:0.98884 val-aucpr:0.90517

[257] ✓ Os print("best_iteration:", bst.best_iteration)
print("best_val_aucpr:", bst.best_score)
# evals_result chứa lịch sử metric
train_curve = evals_result["train"]["aucpr"]
val_curve = evals_result["val"]["aucpr"]

best_iteration: 473
best_val_aucpr: 0.9055037254150373
```

We trained XGB with `n_estimators=2000`, `learning_rate=0.05` and early stopping (patience=50) on the Validation set using AUC-PR. Training stopped at 473 boosting rounds (Val AUC-PR 0.9055). Predictions on Val/Test use the best iteration only; the operating threshold `t*` is selected on Validation and then fixed for Test.

We trained the XGBoost model using `n_estimators=2000`, `learning_rate=0.05`, and early stopping with a patience of 50 rounds. The evaluation metric used was AUC-PR on the validation set. The training process terminated at the 473rd boosting round, where the validation AUC-PR reached its peak value of 0.9055.

During this training, the `best_iteration` was selected based on the highest validation AUC-PR. This best iteration was then fixed and used to generate predictions on the test set. As a result, the final performance on the test set (e.g., F1 Score) might slightly differ from the validation performance, due to distributional differences or class imbalances between the validation and test splits.

This approach ensures that the model is not overfitted on the training data and helps provide a more reliable generalization estimate for unseen data.

Appendix – Artifact (included with the submission)

✓ Artifact for the final model (Appendix)

[48]
✓ Os

```
# B1 - Save artifacts + self-check (lite)
import os, json, zipfile, joblib
import numpy as np, pandas as pd
from sklearn.metrics import average_precision_score, roc_auc_score, f1_score

os.makedirs("artifacts", exist_ok=True)

try:
    p_test = proba_test_xgb
except NameError:
    p_test = proba_test

# ===== 2) METRICS TEST FUNCTION (applied to VALIDATION) =====
metrics_xgb = {
    "PR-AUC": float(average_precision_score(y_test, p_test)),
    "ROC-AUC": float(roc_auc_score(y_test, p_test)),
    "F1@t": float(f1_score(y_test, (p_test >= float(t_star_xgb)).astype(int))),
    "t": float(t_star_xgb),
}

# ===== 3) Save ARTIFACTS =====
pd.Series(p_test, name="proba_xgb_test").to_csv("artifacts/proba_xgb_test.csv", index=False)
try:
    joblib.dump(xgb_model, "artifacts/xgb_model.pkl")
except NameError:

    try:
        bst.save_model("artifacts/xgb_model.json")
    except Exception:
        pass

    with open("artifacts/metrics.json", "w") as f:
        json.dump({"XGB": {"TEST": metrics_xgb}}, f, indent=2)

    with zipfile.ZipFile("artifacts.zip", "w", zipfile.ZIP_DEFLATED) as z:
        for fn in os.listdir("artifacts"):
            z.write(os.path.join("artifacts", fn), arcname=fn)
    print("✓ Saved artifacts.zip")

# ===== 4) SELF-CHECK =====
expected_xgb = {"PR-AUC": 0.9045, "F1@t": 0.8375}
assert abs(metrics_xgb["PR-AUC"] - expected_xgb["PR-AUC"]) <= 0.001
assert abs(metrics_xgb["F1@t"] - expected_xgb["F1@t"]) <= 0.002
print("✓ Self-check passed ")
```

✓ Saved artifacts.zip
✓ Self-check passed

```
import json, pandas as pd, numpy as np
from sklearn.metrics import average_precision_score, roc_auc_score, f1_score, confusion_matrix

# 1) Read artifacts
proba = pd.read_csv("artifacts/proba_xgb_test.csv")["proba_xgb_test"].to_numpy()
res = json.load(open("artifacts/metrics.json"))["XGB"]["TEST"]
t = float(res["t"])

pr = average_precision_score(y_test, proba)
roc = roc_auc_score(y_test, proba)
f1 = f1_score(y_test, (proba >= t).astype(int))

print(f"Recomputed : PR-AUC={pr:.4f} ROC-AUC={roc:.4f} F1@t={f1:.4f} (t={t:.2f})")
print(f"From metrics: PR-AUC={res['PR-AUC']:.4f} ROC-AUC={res['ROC-AUC']:.4f} F1@t={res['F1@t']:.4f}")

# 3) Self-check with a small threshold
TOL = {"PR-AUC": 0.001, "ROC-AUC": 0.001, "F1@t": 0.002}
assert abs(pr - res["PR-AUC"]) <= TOL["PR-AUC"], "PR-AUC lệch quá ngưỡng"
assert abs(roc - res["ROC-AUC"]) <= TOL["ROC-AUC"], "ROC-AUC lệch quá ngưỡng"
assert abs(f1 - res["F1@t"]) <= TOL["F1@t"], "F1 lệch quá ngưỡng"
print("✅ Metrics match with metrics.json")

# confusion matrix
cm = confusion_matrix(y_test, (proba >= t).astype(int))
print("Confusion matrix (TN FP; FN TP):\n", cm)
```

```
Recomputed : PR-AUC=0.9045 ROC-AUC=0.9467 F1@t=0.8375 (t=0.67)
From metrics: PR-AUC=0.9045 ROC-AUC=0.9467 F1@t=0.8375
✔ Metrics match with metrics.json
Confusion matrix (TN FP; FN TP):
[[5014 52]
 [ 359 1059]]
```

```
import hashlib
h=hashlib.sha256()
with open("artifacts.zip","rb") as f:
    for ch in iter(lambda: f.read(1<<20), b''): h.update(ch)
print("artifacts.zip SHA256 =", h.hexdigest())
```

```
artifacts.zip SHA256 = e9e57d75c9af94d65e1ad8adfc7bc21bbb7deae1a0c0fea866a0f7b2c872dbe5
```

- `metrics.json` — PR-AUC, ROC-AUC, F1@t, and the threshold $t^* \wedge t^*$ used on the test set.
- `proba_xgb_test.csv` — XGB predicted probabilities on the test set.
- `xgb_model.json` — the trained XGBoost model (Booster).
- `feature_importance_gain.csv` — global feature importance (gain) from XGB.

```
artifacts.zip                                SHA256                                     =
e9e57d75c9af94d65e1ad8adfc7bc21bbb7daea1a0c0fea866a0f7b2c872dbe5
```